# Учитесь руками, ускоряйтесь ИИ: маршрут к senior

*Отчёт подготовлен 26 октября 2026 года. Цены и лимиты инструментов меняются ежемесячно: всё, что помечено как «не подтверждено» или «сторонний источник», перед покупкой проверьте на официальных страницах.*

В 2026 году быстрее всего до блестящего владения доводит схема «ИИ как репетитор, а не исполнитель». В рандомизированном исследовании Anthropic (январь 2026) младшие Python-разработчики осваивали новую асинхронную библиотеку. Те, кто работал с ИИ-ассистентом, набрали **50% против 67%** на тесте без ИИ (Cohen's d = 0,74). Значимого выигрыша во времени у них не было, а сильнее всего пострадала **отладка**. Участники, которые задавали ИИ только концептуальные вопросы, сохранили почти весь результат: 65% (Anthropic). Ту же картину дают полевой эксперимент в PNAS (минус 17% на экзамене после работы с обычным GPT-4) и RCT в TUM: баллы на упражнениях выше, прирост знаний тот же. Поэтому ядро плана — методики с самыми большими доказанными эффектами: извлечение из памяти и интервальное повторение (g ≈ 0,5–0,6), самообъяснение (g ≈ 0,55), разобранные примеры (g ≈ 0,48) с постепенным переходом к самостоятельным задачам и цикл «прочитай → отследи выполнение → измени → напиши». ИИ используется только в обучающих режимах: Claude Code Learning с `TODO(human)`, ChatGPT Study Mode, Gemini Guided Learning, NotebookLM. По моей оценке, до уровня «senior-знаний» нужно около **1900 часов** целенаправленной работы. Это вывод, а не измеренная величина: примерно 4 года при 10 ч/нед., 2,75 года при 15 ч/нед. и 1,7 года при 25 ч/нед. Само звание senior сверх этого требует обратной связи из реальной работы: ревью, инцидентов, ответственности за дизайн. Ранее предложенный план (четыре этапа, Rust вторым языком) подтверждается с тремя поправками. Навыки проверки (отладка, тесты, ревью, безопасность) должны идти сквозной линией с первого дня. CS-фундамент нужно перемежать с проектами, а не выделять в отдельный блок. Rust стоит начинать только после выхода из второй фазы. На фазы 1–2 хватает бюджета $0, а лучшие $20 в месяц — это Claude Pro ради Claude Code. Бесплатные ИИ-агенты в 2026 году — ненадёжный фундамент для плана. Gemini CLI для бесплатных аккаунтов заменён Antigravity CLI, Copilot перешёл на AI-кредиты и на два месяца закрывал регистрацию, в ChatGPT Free появилась реклама.

## Делегирование ИИ стоит 17 пунктов понимания, концептуальные вопросы — почти ничего

Главный научный результат 2025–2026 годов: **как вы работаете с ИИ во время учёбы, сильнее влияет на усвоение, чем то, работаете ли вы с ним вообще**. В RCT Anthropic (Shen и Tamkin, arXiv 2601.20245) участвовали 52 инженера, которые больше года писали на Python каждую неделю. Им дали 35 минут на две функции в незнакомой библиотеке Trio, а затем тест без ИИ. Группа с ИИ закончила лишь примерно на 2 минуты быстрее (p = 0,391, незначимо), но тест написала на 17 пунктов хуже (Anthropic). Решающими оказались паттерны взаимодействия. Полное делегирование дало 39%, «постепенная зависимость» (сначала вопросы, потом весь код отдаётся ИИ) — 35%, **отладка руками ИИ — 24%**. «Сгенерировать, а затем расспросить» дало **86%**, «код

вместе с объяснением» — 68%, «только концептуальные вопросы, ошибки исправляю сам» — 65% (Anthropic). У этого исследования есть оговорки: в каждом кластере всего 2–7 человек, поэтому данные корреляционные, а тест проводился сразу после задания. Авторы при этом ожидают, что агентные инструменты вроде Claude Code дадут **ещё более сильный** эффект, чем чат-ассистент в боковой панели.

Это не единичный результат. Во всех найденных рандомизированных исследованиях, где отдельно измеряли результат при работе с ИИ и обучение без него, обнаруживается **разрыв между выполнением и обучением**:

| Исследование | Дизайн | Результат | Что это значит для вас |
|---|---|---|---|
| Bastani et al., PNAS 2025 | ~1000 школьников, математика, полевой RCT | Обычный GPT-4: +48% на практике, **−17% на экзамене**. GPT-тьютор с подсказками вместо ответов устранил вред, но выигрыша не дал (Wharton) | Хорошие результаты с ИИ не означают навыка |
| Bassner et al., TUM, N = 275 | CS1, задание на конкурентность, 3 группы | Оба варианта ИИ подняли баллы и снизили когнитивную нагрузку, но **не улучшили прирост знаний и понимание кода**. Авторы называют это «ловушкой комфорта» (TUM) | Лёгкость учёбы обманчива |
| Lehmann et al. | 2 предрегистрированных эксперимента, Python | Замещение (ИИ решает) даёт больше тем при меньшем понимании каждой. Дополнение (ИИ объясняет) даёт больше понимания. ИИ **увеличивает разрыв** между слабыми и сильными (arXiv) | Новичку дисциплина нужнее всего |
| Prather et al., ICER 2024 | 21 сессия с новичками, eye-tracking | Сильные ускоряются, а у слабых ИИ усугубляет метакогнитивные трудности и создаёт **иллюзию компетентности** (arXiv) | Опасность «я же сделал CRM» |
| METR, 2025 | 16 опытных мейнтейнеров, 246 задач | С ИИ работа шла на **19% дольше**, а субъективно казалась на **20% быстрее** (METR) | Ощущению скорости верить нельзя |

Данные о продуктивности не противоречат этим выводам, а дополняют их. На ограниченных, чётко поставленных задачах ускорение реально: в трёх полевых RCT с 4867 разработчиками Copilot дал **+26% завершённых задач**, причём менее опытные выигрывали больше (SSRN), а в Google — около 21% экономии времени (arXiv). Продолжение METR 2026 года показывает вероятное ускорение с инструментами конца 2025 года (центральные оценки −18% и −4% времени). Однако оба доверительных интервала включают ноль, а 30–50% разработчиков не сдавали задачи, которые не хотели делать без ИИ (METR). Отсюда главный вывод для вас. Вы уже делаете проекты (CRM, FileMerger, TopoFlow AI, Topological Laser) с помощью ИИ, и это ценный актив: есть домен, мотивация и понимание задач. Но это одновременно ровно тот сценарий, в котором исследования фиксируют высокую производительность при низком усвоении. Готовые проекты сами по себе не доказывают владения языком. Владение доказывает только регулярная **проверка без ИИ**.

## Извлечение, интервалы и трассировка кода дают эффект g ≈ 0,5

Для общих техник обучения доказательная база лучше, чем для большинства советов о программировании. Dunlosky et al. (2013) отнесли к методам «высокой полезности» только два:

**самопроверку и распределённую практику**. Перечитывание, конспектирование и выделение получили оценку «низкая полезность» (APS). Ниже собраны размеры эффектов с пометкой о степени уверенности. Пометка «библиографически» означает, что цифра взята из известной статьи, но в рамках исследования не перепроверялась.

| Метод | Эффект | Источник | Применение к Python |
|---|---|---|---|
| Самопроверка, извлечение из памяти | g ≈ 0,5–0,6 (библиографически); с обратной связью эффект сильнее | Adesope 2017, Rowland 2014 | Писать код по памяти, предсказывать вывод, карточки Anki |
| Интервальное повторение | Устойчиво лучше «зубрёжки»; оптимальный интервал растёт со сроком хранения | Cepeda 2006 (библиографически) | FSRS в Anki, 15–20 мин в день |
| Самообъяснение | **g = 0,55**, 64 работы, эффект сохраняется при контроле времени | Bisra 2018 | Одна фраза «что делает функция» после чтения кода |
| Разобранные примеры | **g = 0,48**; добавление подсказок к самообъяснению *снизило* эффект | Barbieri 2023 | Для новых тем (asyncio, SQL-планы); затем убирать опоры |
| Чередование | **g = 0,42**; лучше всего работает для похожих, легко путаемых категорий | Brunmair & Richter 2019 | list/dict/set, рекурсия/итерация, threads/async/processes вперемешку |
| Продуктивная неудача | **g = 0,36** (0,37–0,58 при точном соблюдении дизайна); на программировании не проверялась | Sinha & Kapur 2021 | Спроектировать кэш или rate limiter *до* чтения LRU и token bucket |

У этих методов есть две важные границы. Первая — **эффект обратной экспертизы**: разобранные примеры помогают новичку, но становятся бесполезными или вредными, когда знаний уже достаточно, и тогда лучше работает самостоятельное решение задач (Kalyuga 2003, библиографически). Из этого следует, что один и тот же режим ИИ («покажи пример») уместен в первый месяц и вреден на десятом. Вторая граница касается «10 000 часов». Метаанализ Macnamara et al. показал, что целенаправленная практика объясняет 26% разброса результатов в играх, 21% в музыке и **менее 1% в профессиях** (APS). Двойное слепое воспроизведение классического исследования скрипачей не подтвердило исходный вывод (PMC). Практика, конечно, важна. Но простой подсчёт часов почти ничего не предсказывает, поэтому **качество часа важнее количества**. Отсюда правило плана: час делегирования ИИ не засчитывается как учебный.

Для программирования есть собственная доказательная база. Многонациональное исследование Lister et al. (2004) показало, что многие студенты после курса не способны надёжно предсказать вывод короткого кода (UTS). Последующие работы установили, что **умение трассировать выполнение и пересказывать код простыми словами вместе предсказывает умение писать код** (DOI, библиографически). На этом построен PRIMM (Predict–Run–Investigate–Modify–Make). В исследовании на 493 школьниках из 13 школ PRIMM-классы показали более высокий результат на итоговом тесте, чем контрольные (QMUL). Задачи Парсонса (собрать программу из перемешанных строк) дают **тот же прирост знаний за меньшее время** (Georgia Tech). Последующие работы, правда, показывают смешанную картину: возможно, Парсонс в основном экономит время, а не добавляет знаний (NSF PAR). Метки подцелей в разобранных примерах в программировании

воспроизводятся непоследовательно (GSU). По code katas контролируемых исследований не найдено, есть только практические отзывы.

Исследования экспертизы объясняют, чем senior отличается по существу. Эксперты хранят знания крупными **смысловыми блоками** (чанками), а новички организуют их по бытовым ассоциациям (McKeithen et al. 1981). Эксперты мыслят «программными планами», то есть типовыми паттернами, и замечают их нарушения (Soloway & Ehrlich 1984, библиографически). Сильные профессионалы тратят пропорционально больше времени на **понимание задачи** и чаще пользуются обратной связью, а стаж слабо связан с результатом (Sonnentag 1998, библиографически). Code review в Microsoft прежде всего передаёт знания и альтернативные решения, а поиск дефектов вторичен (Bacchelli & Bird 2013). Контролируемых исследований о том, как *обучать* архитектурному суждению, не найдено. Поэтому всё, что в плане касается архитектуры, — вывод из общих механизмов: прогноз, затем обратная связь, затем рефлексия.

Карточки работают для декларативного знания: синтаксиса, API, сложности операций, компромиссов. Процедурный навык они не создают. Майкл Нильсен тратит на повторение 15–20 минут в день и советует делать карточки атомарными, брать их из собственного кода и заводить карточку, только если факт «стоит» около 10 минут будущего времени (Nielsen). Дерек Сиверс назвал интервальное повторение языка самым полезным приёмом за 14 лет в программировании (sive.rs). Это свидетельство практика, а не испытание. Планировщик FSRS в бенчмарке своих разработчиков обходит SM-2 примерно в 97% коллекций, но эта цифра известна только из форума (Anki forum).

## Рынок 2026 года доплачивает за проверку, а не за генерацию

Рынок подтверждает, что ваша цель — блестящее владение, а не просто быстрая работа — экономически верна. По данным ADP до июня 2026 года, занятость 22–25-летних в профессиях, подверженных ИИ, на **19% ниже** контрфактического тренда. Опытных работников это не затронуло, а механизм — сокращение найма молодых, не увольнения. Авторы прямо называют результат описательным, а не причинным (Stanford DEL). Вакансии в разработке с февраля 2025 года выросли почти на 15%, но **71% чистого прироста пришёлся на senior-роли, 37% — на вакансии с «AI» в названии** (Indeed Hiring Lab). Медианный доход в США (Levels.fyi, 2025): entry $155K, senior $312K, staff $457K. Быстрее всего растёт staff: +7,5% против +1,6% у entry (Levels.fyi). Для удалённой работы из Восточной Европы ориентир senior — около $80–120K, на 40–50% ниже американского (Lemon.io). Данных по Азербайджану и СНГ за 2026 год не найдено.

Самые ценные навыки — ровно те, которые ИИ размывает. В опросе Stack Overflow 2025 года **66%** разработчиков назвали главной проблемой «почти правильные» решения ИИ, 45% — то, что отладка ИИ-кода занимает больше времени, и лишь около 33% доверяют точности ИИ (SO 2025). GitClear фиксирует рост дублирования кода примерно в 8 раз и падение доли рефакторинга с ~25% до менее 10%. Это наблюдательные данные вендора (DevClass). DORA 2025 описывает ИИ как «усилитель» сильных и слабых сторон команды (InfoQ). Следовательно, узкое место смещается к **верификации**: отладке, чтению чужого кода, тестам, ревью, проектированию и безопасности. Базовый стек по данным SO 2025: Python (57,9%, +7 п.п.), Docker (71,1%, +17 п.п., рекордный рост), PostgreSQL (55,6%) и облако (SO 2025).

Опыт в кибербезопасности — ваше главное конкурентное преимущество. 59% специалистов по безопасности сообщают о критической нехватке навыков, и самые востребованные из них — **ИИ**

**(41%), облачная безопасность (36%) и безопасность приложений (28%)** (MeriTalk). При этом 81% разработчиков беспокоит безопасность данных при работе ИИ-агентов (SO 2025). Пересечение «Python + LLM-приложения + безопасность» (prompt injection, права инструментов агента, утечки данных, threat modeling) — это ниша, где ИИ дополняет работника, а не замещает его. По данным Стэнфорда, именно там занятость стабильна или растёт. Привычки безопасника (threat modeling, принцип наименьших привилегий) — это ещё и компетенции архитектора, поэтому они засчитываются дважды. Архетипы StaffEng (Tech Lead, Architect, Solver, Right Hand) показывают, что «архитектор» — это владение технической областью и связь с бизнесом, а не рисование диаграмм (StaffEng).

## Rust — верный второй язык, но не раньше конца второй фазы

Для вашего профиля (производительность, научные симуляции, ИИ, безопасность) данные подтверждают выбор Rust. Быстрый слой современной Python-экосистемы всё больше пишется на Rust: uv, Ruff, ty, polars, pydantic-core, токенизаторы Hugging Face, tiktoken. Rust подключается к Python через **PyO3 и maturin** (PyO3). Доля Rust среди бинарных расширений Python за год выросла **с 27% до 33%** (heise). Rust — самый «admired» язык в SO 2025 (72%) (SD Times). OpenAI объявила о покупке Astral (uv, Ruff, ty) 19 марта 2026 года (SiliconANGLE). Это знак того, что связка Python и Rust стала стратегической инфраструктурой. Для Topological Laser и TopoFlow естественный сценарий такой: горячие циклы (решатели ODE/PDE, распространение пучка, топологические инварианты на сетке) переносятся в Rust-расширение и вызываются из NumPy.

| Язык | За | Против | Когда выбирать |
|---|---|---|---|
| **Rust** | PyO3/maturin, научные ядра, безопасность памяти, инструменты для security, 72% admired | Крутая кривая: 2–4 месяца борьбы с borrow checker, 6–12 месяцев до уверенного async и lifetimes *(оценка)* | Ваш профиль: физика, симуляции, ИИ-инфраструктура, security |
| Go | Примерно в 3–4 раза больше вакансий, чем у Rust *(слабый источник)*; облако и DevOps; быстро осваивается | Мало что добавляет к численным расчётам на Python | Если целитесь в DevOps, SRE или cloud backend (tech-insider) |
| TypeScript | №1 на GitHub по числу контрибьюторов с августа 2025 года (Octoverse) | Ничего не даёт для производительности и симуляций | Позже и только для веб-интерфейсов (UI для CRM, дашборды TopoFlow) |
| C/C++ | CUDA, унаследованные физические коды, reverse engineering и эксплойты | Безопасно писать на C/C++ учатся годами | Достаточно уровня чтения, который естественно появляется после CS:APP и Nand2Tetris |

Поправка к прежнему совету касается **сроков**. Если начинать Rust параллельно с базовым Python, придётся одновременно осваивать две системы типов, две модели памяти и два набора идиом. Цена этого — потерянная консолидация. Это вывод, прямых данных нет. Rust оправдан, когда выполнены критерии выхода из фазы 2: вы уверенно пишете типизированный, протестированный Python и понимаете модель «имена ссылаются на объекты». Тогда сравнение с ownership в Rust само становится учебным материалом. Ресурсы бесплатные: Rust Book, Rustlings, руководство PyO3. На продвинутом уровне — Rust for Rustaceans (Rust Book, Rustlings).

## Четыре фазы и ~1900 часов: от 1,7 до 4 лет до уровня senior-знаний

Ни одно рецензируемое исследование не связывает число часов с уровнем senior. Поэтому все объёмы ниже — **инженерная оценка**, собранная из ориентиров. Один вузовский курс (~100+ часов) даёт хрупкую компетентность (Lister). Буткемпы длятся 3–6 месяцев полного дня (Course Report). teachyourselfcs советует примерно 100–200 часов на предмет; эта цифра не перепроверялась (teachyourselfcs). Учтено и ваше преимущество: у вас уже есть опыт с проектами и предметной областью, поэтому первая фаза короче, чем у абсолютного новичка.

### Фазы, содержание и режим ИИ

| Фаза | Цель | Содержание | Часы *(оценка)* | Режим ИИ |
|---|---|---|---|---|
| **1. Фундамент руками** | Надёжная ментальная модель Python, умение писать с чистого листа | CS50P (cs50.harvard.edu/python), официальный tutorial, Exercism Python (50+ задач), PRIMM для каждой новой конструкции, трассировка в pythontutor/pdb, Anki с первого дня, Git | **300** | Только репетитор: Study Mode, Guided Learning, Claude Learning mode, CS50 Duck. **Автодополнение Copilot выключено.** Генерация кода не используется |
| **2. Инженерия на своих проектах** | Писать идиоматичный, типизированный, протестированный код; читать чужой | Fluent Python 2e, Effective Python 3e (125 советов, Python 3.13), Python Testing with pytest; uv, Ruff, pyright/ty, FastAPI, Pydantic v2, SQLAlchemy 2, Docker, PostgreSQL, CI. **Сначала прочитать и объяснить** существующий ИИ-код FileMerger и CRM, затем переписать с тестами. Параллельно CS50x и Composing Programs | **400** | Claude Code в стиле **Learning** (`TODO(human)`); «сгенерируй, затем расспроси» только для шаблонного кода; правило «сначала отлаживаю сам» |
| **3. CS-фундамент + Rust** | Понимать, что происходит под Python; второй язык | Ядро teachyourselfcs, привязанное к вашим целям: CS:APP или Nand2Tetris (память, C), OSTEP (ОС), Kurose & Ross (сети, это для security), CMU 15-445 (БД), алгоритмы. CodeCrafters (свой Redis или HTTP-сервер). Rust Book, Rustlings, **PyO3-расширение для ядра симуляции Topological Laser с бенчмарками** | **600** | Explanatory для чтения чужих кодовых баз (CPython, polars). Для Rust первые ~100 часов без автодополнения. Продуктивная неудача: сначала свой дизайн, потом канонический |

| Фаза | Цель | Содержание | Часы *(оценка)* | Режим ИИ |
|---|---|---|---|---|
| **4. Архитектура, системы, AI-инженерия** | Суждение о компромиссах, проектирование систем, безопасный ИИ | Cosmic Python (бесплатно), A Philosophy of Software Design, Fundamentals of Software Architecture 2e (2025), **DDIA 2e (2026)** (Kleppmann), Software Engineering at Google (бесплатно), AI Engineering (Huyen), Raschka «LLM from scratch». ADR и design docs для TopoFlow AI; LLM-приложение с набором эвалов и threat model; PR в open source | **600** | Агенты свободно используются для задач, которые вы уже понимаете (Jules, Copilot agent, Codex). **Ревью их PR как тренировка.** OpenRouter, NVIDIA build и AI Studio для LLM-приложений |
| Итого | | | **~1900** | |

## Расчёт сроков для 10, 15 и 25 часов в неделю

Расчёт исходит из **46 эффективных недель в году**: отпуска, болезни и сбои уже вычтены.

| Фаза (часы) | 10 ч/нед. (~460 ч/год) | 15 ч/нед. (~690 ч/год) | 25 ч/нед. (~1150 ч/год) |
|---|---|---|---|
| 1. Фундамент (300) | ~30 нед. ≈ 8 мес. | ~20 нед. ≈ 5 мес. | ~12 нед. ≈ 3 мес. |
| 2. Инженерия (400) | ~40 нед. ≈ 10 мес. | ~27 нед. ≈ 7 мес. | ~16 нед. ≈ 4 мес. |
| 3. CS + Rust (600) | ~60 нед. ≈ 16 мес. | ~40 нед. ≈ 10 мес. | ~24 нед. ≈ 6 мес. |
| 4. Архитектура (600) | ~60 нед. ≈ 16 мес. | ~40 нед. ≈ 10 мес. | ~24 нед. ≈ 6 мес. |
| **Всего** | **≈ 4,1 года** | **≈ 2,75 года** | **≈ 1,7 года** |
| Когда начинать Rust | ~18-й месяц | ~12-й месяц | ~7-й месяц |

У этой таблицы три важные оговорки. Первая: **25 ч/нед. не сжимают путь линейно**. Эффект интервалов означает, что консолидация требует календарного времени: оптимальный промежуток между повторениями растёт с желаемым сроком хранения (Cepeda 2008, библиографически). Поэтому реалистичный срок при 25 ч/нед. ближе к 2 годам, чем к 1,7. Вторая: на выходе получается «senior по знаниям» или сильный mid, а **суждение senior формируется годами разнообразной практики с обратной связью**: ревью, инциденты, ответственность за дизайн. Распространённое правило «senior — это 5–8 лет стажа» в ходе исследования не нашло подтверждения в источниках, поэтому опирайтесь не на годы, а на критерии ниже. Третья: оптимальный сценарий для вас — **15 ч/нед.** Он даёт полноценные сессии без выгорания и оставляет время на консолидацию. 10 ч/нед. — минимум, при котором ещё сохраняется непрерывность. 25 ч/нед. оправданы, если это основная деятельность.

## Шаблон недели (15 ч/нед.)

| День | Блок | Время | Содержание |
|---|---|---|---|
| Ежедневно (6 дней) | Anki (FSRS, целевое удержание 0,9) | 15–20 мин | Карточки из *собственных* ошибок, ревью и прочитанных документов (Nielsen) |

| День | Блок | Время | Содержание |
|---|---|---|---|
| Пн, Ср, Пт | Ядро | по 2 ч | 15 мин: написать по памяти то, что изучено на прошлом занятии (извлечение) → 50 мин: новая тема по PRIMM или разобранный пример → 55 мин: задачи **без ИИ** (Exercism, Парсонс, трассировка), чередуя похожие типы задач |
| Вт, Чт | Проект и чтение | по 1,5 ч | 60 мин: работа над своим проектом (в фазе 2 и далее — Claude Code Learning). 30 мин: чтение хорошего кода (stdlib `collections` / `functools`, requests) с пересказом каждой функции одной фразой |
| Сб | Глубокая работа | 3 ч | 2 ч: проект и журнал отладки («гипотеза → проверка → результат»). 45 мин: **еженедельная контрольная без ИИ**. 15 мин: замер времени, прогноз против факта |
| Вс | Рефлексия | 1 ч | Объяснение одной концепции «для джуна» (Feynman, ADR или пост) и сверка с источником; план недели |
| **Итого** | | **≈ 15 ч** | |

Как масштабировать. При **10 ч/нед.** ядро сокращается до трёх сессий по 1,5 ч, проектный блок до одного раза в неделю, субботний блок до 2 ч, а Anki остаётся ежедневным: его нельзя сокращать, потому что он и есть интервальность. При **25 ч/нед.** добавляются два ядра по 2 ч и проектный блок 3 ч в будние дни, а в фазах 3–4 — отдельный еженедельный слот на 3 ч для CS-курса или книги. Примерное распределение времени по типам деятельности:

| Тип деятельности | Фаза 1 | Фаза 2 | Фаза 3 | Фаза 4 |
|---|---|---|---|---|
| Упражнения и задачи без ИИ | 45% | 25% | 25% | 10% |
| Проект (с ИИ в обучающем режиме) | 10% | 40% | 30% | 40% |
| Чтение (книги, чужой код) | 20% | 15% | 30% | 30% |
| Извлечение и повторение (Anki, контрольные) | 15% | 10% | 8% | 7% |
| Письменные объяснения, ADR, ревью | 10% | 10% | 7% | 13% |

## Метрики выхода из каждой фазы

Все пороги ниже — мои выводы. Они опираются на два доказанных принципа: трассировка и пересказ предсказывают умение писать код (Lister), а результат с ИИ не предсказывает результат без него (Bastani). **Каждая метрика проверяется без ИИ.**

| Фаза | Контрольные точки (все без ИИ) |
|---|---|
| 1 | ≥ 90% правильных предсказаний вывода для фрагментов по 15–25 строк (изменяемость, области видимости, аргументы по умолчанию, генераторы). Задачу Exercism средней сложности решаете за 30 мин или быстрее. С чистого листа пишете CLI-утилиту на 300 строк с тестами за выходные. Объясняете вслух модель «имена → объекты». Финальный проект CS50P. Удержание в Anki ≥ 85% |
| 2 | Переписанный FileMerger или CRM: тесты (ориентир ≥ 80% покрытия ключевой логики), чистые Ruff и pyright, CI, Docker. Три специально внесённые ошибки находите сами меньше чем за час. Незнакомый модуль на ~2000 строк описываете на одной странице. Ошибка ваших оценок времени снижается от месяца к месяцу |

| Фаза | Контрольные точки (все без ИИ) |
|---|---|
| 3 | Выполнены лабораторные CS:APP или Nand2Tetris. Свой Redis или HTTP-сервер (CodeCrafters или build-your-own-x). LRU-кэш и rate limiter спроектированы сами, а затем сверены с каноническими. Rustlings пройдены. **PyO3-ядро симуляции с воспроизводимым бенчмарком против NumPy-версии.** Объясняете сложность и выбор структур данных |
| 4 | 3+ ADR, сверенных с фактическим исходом через 1–3 месяца. Design doc для TopoFlow AI. LLM-приложение с набором эвалов, задокументированными сбоями и threat model по prompt injection. 3+ принятых PR в чужие open-source проекты. 20+ отревьюенных PR, включая PR от агентов. Пробное system design-интервью |

Раз в месяц полезно проводить **90-минутный экзамен без ИИ** на новой задаче своего уровня и сравнивать его с предыдущим. Ещё одна метрика — доля учебного времени, когда код писали вы, а не ИИ. В фазе 1 она должна быть около 100%, в фазе 2 — не ниже ~60%. Падение результатов без ИИ при росте результатов с ИИ — тот самый «разрыв» из исследований, и это сигнал откатиться на шаг назад.

### Проверка прежнего плана из четырёх этапов

| Прежний совет | Вердикт | Поправка по данным |
|---|---|---|
| Базовый Python руками | **Подтверждён** | «Руками» не значит «без ИИ». Концептуальные вопросы почти не вредят (65% против 67%). Нужно добавить трассировку, PRIMM и Anki: перечитывание и просмотр видео — методы «низкой полезности» |
| Переписать свои проекты | **Подтверждён с уточнением** | Начинать после выполнения метрик фазы 1. Сначала прочитать и объяснить ИИ-код (это тренирует чтение чужого кода — ключевой навык senior), затем переписать с тестами в режиме Learning |
| CS-фундамент | **Подтверждён, но иначе по форме** | Не отдельный блок, а чередование с проектами с фазы 2 (интервалы + чередование). Ядро предметов выбирается по целям: ОС и сети для security, архитектура и численные методы для симуляций |
| Архитектура | **Подтверждён, но недостаточен** | Книг мало: суждение формируется в циклах «прогноз → исход» (ADR, ревью, open source), а не чтением. Верификацию и безопасность нужно вести сквозной линией с фазы 1 |
| Rust вторым языком | **Подтверждён для вашего профиля** | Начинать в фазе 3, а не параллельно с основой. Go — только при цели DevOps или cloud. C — на уровне чтения |

## Инструменты распределяются по задачам, а не по брендам

Все три крупных вендора к 2026 году встроили «режим репетитора»: Claude Learning mode, ChatGPT Study Mode и Gemini Guided Learning. Но только Claude Code в стиле **Learning** *заставляет* писать код внутри реального репозитория. В местах с настоящим проектным решением (обработка ошибок, структура данных, бизнес-логика) он оставляет `TODO(human)`, останавливается и ждёт (Claude Code docs). Контролируемых испытаний обучающих режимов не найдено. Есть лишь косвенный аргумент: GPT-тьютор с ограничениями в PNAS устранил вред от ИИ, но выигрыша не дал (Wharton). Поэтому обучающий режим — это страховка, а не ускоритель.

## Ключевые изменения 2026 года, которые ломают старые советы

| Изменение | Дата | Следствие для плана | Уверенность |
|---|---|---|---|
| **Gemini CLI заменён Antigravity CLI** для бесплатных пользователей и Google One (geminicli.com) | 18.06.2026 | Не стройте план на Gemini CLI. Antigravity CLI закрытый (Go), бесплатная квота неясна: ~20 запросов в день или недельный лимит по вычислениям (The Register) | Высокая (замена); низкая (квоты) |
| Antigravity IDE: Claude и GPT-OSS убираются из бесплатного тарифа (Preuve) | 02.11.2026 | Бесплатно остаются только модели Gemini | Средняя (один источник) |
| **Copilot перешёл на AI-кредиты**; регистрации Student, Pro и Pro+ были закрыты (GitHub Changelog) | 20.04 – ~17.06.2026 | Free: 2000 автодополнений, только Auto, без агента. Student: агент есть, но выбор модели только через Auto. Opus убран из Pro | Высокая (официально); объём кредитов Free и Student не опубликован (сторонний источник: 200 для Student) |
| ChatGPT: реклама в Free и Go; Free даёт безлимит текста, но только на малой GPT-5.6 Luna (Engadget) | Февраль и август 2026 | Study Mode бесплатен, но модель слабая. В Plus с 25.08.2026 вернулось 5-часовое окно Codex | Средняя |
| AI Studio: Pro-модели убраны из бесплатного API; дневной лимит 3.5 Flash спорный: 20 или 1500 RPD (Tilnote) | Апрель 2026 | Не рассчитывайте на большие бесплатные объёмы | Низкая (цифры) |
| Firebase Studio закрывается (Storyboard18) | 22.03.2027 | Не используйте | Высокая |
| OpenAI покупает Astral (uv, Ruff, ty) (The New Stack) | 19.03.2026 | Инструменты обещано поддерживать. ty всё ещё 0.0.x | Высокая (анонс); закрытие сделки не подтверждено |
| Manus: покупка Meta → блокировка Китаем → снова независимая компания (PricingSaaS) | Декабрь 2025 – август 2026 | Ненадёжный тариф; инструмент второстепенный | Низкая |

## Таблица распределения инструментов

| Инструмент | Задача в плане | Бесплатно / платно (окт. 2026) | Когда НЕ использовать |
|---|---|---|---|
| **Claude (claude.ai), Learning mode** | Сократический разбор концепций, интерактивные Artifacts (визуализация алгоритмов) | Free: Sonnet и Haiku, 5 Projects, Artifacts; точные лимиты сообщений не опубликованы (pricing) | Не просите готовые решения упражнений фазы 1 |

| Инструмент | Задача в плане | Бесплатно / платно (окт. 2026) | Когда НЕ использовать |
|---|---|---|---|
| **Claude Code: Learning / Explanatory / свой стиль** | Главный инструмент фаз 2–4: проект с `TODO(human)`; Explanatory — для чтения чужих кодовых баз | Только Pro: 20/*мес*.(17 при оплате за год), общий пул с чатом, окно 5 ч + недельный лимит; обучающие стили расходуют больше токенов (docs) | В фазе 1; в стиле Default для учебных задач; для отладки до собственной попытки |
| **ChatGPT Study Mode** | Ежедневные вопросы по теории, самопроверка, устные объяснения | Free: безлимит текста на GPT-5.6 Luna, с рекламой; Go ~$8; Plus $20 (TechCrunch) | Для сложной архитектуры на Free-модели; не держите вторую подписку за $20 параллельно с Claude Pro |
| Codex (CLI, облако) | Фаза 4: агентные задачи и ревью его PR | Входит во все планы, включая Free; лимиты не опубликованы (OpenAI Help) | В фазах 1–2 |
| **Gemini Guided Learning** | Визуальные пошаговые объяснения алгоритмов и CS-теории | Бесплатно (TechCrunch) | Как замена собственным попыткам решить задачу |
| **NotebookLM** | Учёба по книгам и документации: квизы и карточки только по вашим источникам, аудиообзоры для повторения | Free, по сторонним данным: ~50 запросов в день, 3 аудиообзора в день, 100 блокнотов (DailyAIFixs) | Аудиообзор вместо чтения — это пассивное восприятие, «низкая полезность». Используйте его только как повтор |
| AI Studio / Gemini API | Первый проект с вызовом LLM API (фаза 4) | Бесплатный тариф, лимиты спорные; данные могут использоваться Google (ai.google.dev) | С конфиденциальными данными (CRM) |
| Antigravity IDE / CLI | Опционально: бесплатная агентная IDE | Бесплатно с «базовыми недельными лимитами»; с 02.11 только Gemini (CloudZero) | Как основа плана: квоты непрозрачны |
| **Jules** | Фаза 4: асинхронный агент. Вы пишете issue и ревьюите его PR — это тренировка ревью | Free: 15 задач в день, 3 одновременно (данные 2025 года) (TechCrunch) | Для задач, которые вы сами пока не умеете решать |
| **GitHub Copilot** | Автодополнение в фазах 2–4; облачный агент (Student, Pro) | Free: 2000 автодополнений, Auto, без агента. Student: бесплатно, с агентом. Pro $10 (1500 кредитов) (Docs) | **Автодополнение выключено в фазе 1** и в первые ~100 часов Rust (это мой педагогический вывод) |
| Codespaces | Облачная среда, если компьютер слабый | 120 core-часов (~60 ч на 2 ядрах) + 15 ГБ в месяц (Docs) | Для долгих симуляций: квота быстро кончится |
| **CS50 Duck** | Репетитор для CS50P и CS50x, «не выдаёт ответы по замыслу» | Бесплатно с CS50x (CS50) | Вне курса CS50 |
| **Exercism** | Практика и **живое менторское ревью** | 100% бесплатно (Exercism) | Самым первым шагом: сначала CS50P |

| Инструмент | Задача в плане | Бесплатно / платно (окт. 2026) | Когда НЕ использовать |
|---|---|---|---|
| NVIDIA build (NIM) | Фаза 4: открытые модели (DeepSeek, Qwen, Nemotron) для RAG и агентов | Бесплатно для прототипов, ~40 RPM без возможности повышения (форум NVIDIA) | Для агентных IDE с множеством вызовов; не для продакшена |
| **OpenRouter** | Фаза 4: единое API, сравнение моделей, маршрутизация, учёт стоимости токенов | `:free`-модели: 20 RPM; 50 запросов в день, или 1000 в день после разовой покупки ≥ $10 (docs) | При отрицательном балансе (ошибки 402) |
| Ollama (локально / облако) | Понимание LLM (квантование, контекст); офлайн-работа с приватными данными | Локально бесплатно; облако с лимитами на 5 ч и неделю (DevToolHub) | Если нет GPU или RAM для локальных моделей |
| Manus | Разовое исследование (сравнение курсов, сбор требований вакансий) | Тариф нестабилен; кредиты быстро сгорают (Moclaw) | Для обучения программированию; Deep Research в Gemini и ChatGPT бесплатно делает то же |
| Cursor / Windsurf | Не нужны | Маленькие бесплатные квоты (сторонние данные) (AgentDeals) | Дублируют связку Copilot + Claude |

## Бюджет

| Сценарий | Состав | Стоимость за год | Что теряется |
|---|---|---|---|
| **$0** | Claude Free (Learning mode), ChatGPT Free (Study Mode), Gemini Guided Learning, NotebookLM, CS50 + Duck, Exercism, Copilot Free, Jules, Codex Free, NVIDIA build, OpenRouter (50/день), Ollama, Codespaces. Бесплатные книги: Cosmic Python, SWE at Google, OSTEP, Composing Programs, Nand2Tetris, Rust Book | $0 | Claude Code с `TODO(human)`. Фазы 1–2 этого почти не ощущают, фазы 3–4 — заметно |
| **~$20/мес.** (рекомендуется) | Всё из 0 + * * *ClaudePro* * * (17/мес. при оплате за год) + разово 10*наOpenRouter* + 3–5*ключевыхкниг*(*FluentPython*, *EffectivePython*, *DDIA2e*, *FoSA2e*; 40–70 за книгу — *оценка*) | ~215–250+ 150–300 на книги | Ничего критичного. Альтернативы за те же $20 — ChatGPT Plus (Codex) или Google AI Pro (квоты Antigravity, Jules ×5) — уступают по обучающей механике |
| **Студент** | Copilot Student (бесплатно, с агентом), GitHub Student Pack (Codespaces, JetBrains), Google AI Pro на 12 месяцев (студенты США, активировать **до 31.12.2026**) или AI Plus (140+ стран) (MacRumors), CodeCrafters бесплатно для некоторых вузов (CodeCrafters) | $0 + опционально Claude Pro | Нужна верификация статуса. **Доступность в вашем регионе не проверена.** После 12 месяцев Google AI Pro продлевается автоматически как платный |

Не стоит платить за Boot.dev (399*вгод*)*uCodeCrafters*( 30–39 в месяц по сторонним данным): бесплатные CS50, Exercism и build-your-own-x покрывают те же цели (Boot.dev, build-your-own-x). Это суждение о соотношении цены и пользы, а не о качестве этих платформ.

### Протокол работы с ИИ по фазам

Протокол выведен из паттернов RCT Anthropic и правил «дополнять, а не замещать» (Lehmann). Действуют четыре правила, все — выводы. Первое: прежде чем спрашивать ИИ, вы пытаетесь решить сами — в фазе 1 не меньше 20–30 минут, в фазе 2 и далее не меньше 45 минут на отладку. Второе: каждую строку сгенерированного кода вы должны уметь объяснить, иначе спрашиваете «почему так» и «какие есть альтернативы» (это паттерн с 86%). Третье: **отладку ИИ не поручаете никогда до собственной гипотезы**, потому что этот паттерн дал 24%. Четвёртое: раз в неделю — контрольная без ИИ. Для чатов подойдёт шаблон по мотивам «The Tutor» Моллик (Northwestern): «Мой уровень — …, цель — …. Сначала проверь, что я уже знаю. Задавай по одному вопросу. Давай подсказки по нарастающей (намёк → псевдокод → фрагмент), полное решение — только по моей явной просьбе. После решения попроси меня объяснить код. Закончи тремя проверочными вопросами». Для Claude Code можно создать собственный стиль в `~/.claude/output-styles/` с `keep-coding-instructions: true` (docs). Например:

```
        ---
        name: Strict Tutor
        description: Репетитор, который не пишет за меня ключевой код
        keep-coding-instructions: true
        ---
        Никогда не пиши больше 10 строк нетривиальной логики подряд: оставляй
TODO(human) и жди.
        Перед любым исправлением ошибки спроси мою гипотезу о причине.
        После каждого моего фрагмента дай одно замечание по идиоматичности и один вопрос
на понимание.
```

## Что доказано, что выведено и что неизвестно

| Утверждение | Статус | Основание |
|---|---|---|
| Делегирование ИИ при изучении нового снижает понимание и отладку | **Доказано (RCT, но малые выборки и короткий срок)** | Anthropic, Bastani, TUM, Lehmann |
| Концептуальные вопросы к ИИ почти не вредят | Доказано слабо (n = 7, корреляционно) | Anthropic |
| Самопроверка, интервалы и самообъяснение работают | **Доказано (метаанализы)**, но не на программировании напрямую | Dunlosky, Bisra |
| Трассировка и чтение кода предсказывают умение писать | Доказано (корреляционно, CS1) | Lister |
| Обучающие режимы ИИ (Learning, Study, Guided) улучшают обучение | **Не доказано**: ограничения только убирают вред | Bastani |
| ~1900 часов и сроки 1,7–4 года | **Вывод**; исследований «часы → уровень» нет | Ориентиры: Lister, teachyourselfcs (не перепроверено) |

| Утверждение | Статус | Основание |
|---|---|---|
| Rust — лучший второй язык для вас | Вывод из данных об экосистеме | PyO3, heise |
| Рынок перекошен в сторону senior и AI-ролей | Описательные данные, не причинные | Indeed, Stanford DEL |
| Как обучать архитектурному суждению | **Неизвестно**: контролируемых исследований нет | Только интервью и сравнения экспертов с новичками |
| Долгосрочная (6+ мес.) деградация навыков при работе с агентами | **Неизвестно** | Лонгитюдных исследований нет |
| Квоты Antigravity, AI Studio, NotebookLM, Codex Free, Copilot Free и Student | Неопределённо: сторонние или противоречивые данные | См. таблицу инструментов |

## Заключение

Самое неочевидное следствие исследований касается вашего главного преимущества: умение быстро собирать проекты с ИИ одновременно служит и активом, и ловушкой. Ощущение прогресса, как показал METR, может иметь противоположный знак по сравнению с реальностью. Поэтому эффективность пути определяется не числом инструментов и не часами, а тем, **как часто вы честно измеряете себя без ИИ**. Еженедельная контрольная и ежемесячный экзамен без ИИ — самая дешёвая и самая недооценённая часть плана.

Второй вывод стратегический. В 2026 году рынок дефицитен не на генерацию кода, а на его **проверку**: отладку, ревью, тесты, безопасность, проектирование. Это именно то, что разрушает делегирование, и то, что усиливает ваш опыт в кибербезопасности. Отсюда тактика, которой не было в прежнем плане: в фазах 3–4 агенты (Jules, Copilot agent, Codex) нужны не как ускоритель, а как **бесконечный поставщик чужого кода для ревью**. Ревью, по данным Microsoft, — главный канал передачи инженерного знания, и теперь его можно тренировать без команды. Связка Python + Rust (PyO3) + безопасный ИИ на ваших физических проектах даёт портфолио, которое ИИ не может подделать: измеренные бенчмарки, threat model, наборы эвалов и ADR, сверенные с исходом.